DSA5208: Scalable Distributed Computing for Data Science

Lecture 9: OpenMP

Anthony Chen

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 OpenMP

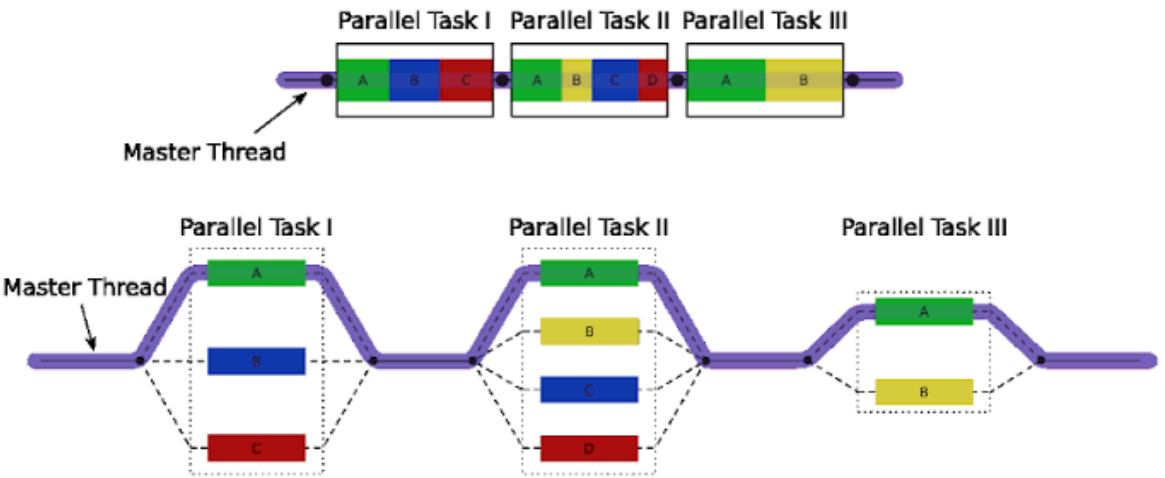
2 C++ Codebase Structure

3 Matrix Multiplication

Threads

A thread is an independent flow of control

- Runtime entity to execute a sequence of instructions
- Processes are executed by one or more threads
- Threads share memory and can have their own private memory



Thread Programming Models

A programming model to use threads should have:

- A way to create and destroy threads
- A way to distribute computation among threads
- A way to coordinate threads
- A way to specify what memory is shared and what memory is private

We will focus on OpenMP as our thread programming model of choice.

OpenMP

OpenMP is a high level thread programming model:

- Users declare the parallelization strategy, especially regarding the creation and distribution of threads
- The compiler then performs the parallelization at compile time

OpenMP utilizes shared memory programming

- Threads communicate by sharing variables
- Unintended sharing leads to race conditions
- Synchronization can be used to avoid race conditions but is expensive
- Better to change memory access patterns

OpenMP Constructs

OpenMP is based on the use of compiler directives (pragmas)

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char* argv[]) {

#pragma omp parallel  // start of the parallel region
    {
        printf("Hello World from thread %d out of %d total\n",
                omp_get_thread_num(), omp_get_num_threads());
    } // End of parallel region
    return 0;
}
```

These directives apply to structured blocks, which is either a single statement (a for loop) or a set of statements enclosed in { }

OpenMP Parallel Regions

The `#pragma omp parallel` construct creates threads.

- When the structured block the construct is tied to ends, the threads are automatically terminated
- Adding `num_threads(N)` allows for manual control over the number of threads
- Adding `if(N>X)` allows for optional parallelization based on runtime state

Consider a For Loop

Parallel regions are nice but clunky. Consider the following serial code:

```
for(int i = 0; i < N; i++) {
    vec[i] += i;
}
```

Using a straightforward parallel region, the equivalent code is

```
#pragma omp parallel
{
    int id, Nthreads, istart, iend;
    id = omp_get_thread_num();
    Nthreads = omp_get_num_threads();
    istart = N*id/Nthreads;
    iend = (id+1)*N/Nthreads;
    for (int i = istart; i < iend; i++) {
        vec[i] += i;
    }
}
```

Unpleasant.

Worksharing Constructs

This is where the enlightened `for` construct enters the picture:

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    vec[i] += i;
}
```

Scheduling Options

The default `#pragma omp parallel for` doesn't expose any information about how work is divided among threads. For more fine grained control, there is the optional `schedule` construct

- `schedule(static)` divides work at compile time
- `schedule(dynamic)` divides work at run time, more scheduling overhead
- `schedule(guide)` is a variant of dynamic scheduling, less scheduling overhead

Sections

Sections are a way to give a different structured block to different threads.

```
#pragma omp parallel
#pragma omp sections
{
#pragma omp section
    // section 1
#pragma omp section
    // section 2
#pragma omp section
    // section 3
}
```

Master Blocks

Sometimes, in a parallel block, you have some code that you only want to run once (perhaps an MPI communication command?). To prevent multiple threads from executing something, use a master construct:

```
#pragma omp parallel
{
// do things here
#pragma omp master
    { /* only executed by one thread */ }
// do some more things
}
```

Can also use the `#pragma omp single` construct.

The Data Environment

As previously mentioned, OpenMP utilizes a shared data parallelism model. This means that

- All variables defined outside of a parallel block are shared
- Variables defined within parallel blocks are private to each thread

Many ways of controlling the data environment.

Data Clauses

There are a number of different specifications:

- `shared(var)` marks `var` as a shared variable
- `private(var)` marks `var` as a private variable
- `firstprivate(var)` marks `var` as a private variable, copies value from outside
- `lastprivate(var)` marks `var` as a private variable, passes value to the outside

Reductions

Recall `MPI_Reduce` and its variants. How can a reduction be performed in parallel with OpenMP?

```
N=1000000;
std::vector<double> vec (N, 1.0);
sum = 0;
#pragma omp parallel for reduction(+:sum)
{
    for (int i = 0; i < N; i++) {
        sum += vec[i];
    }
}
```

A bit more complicated than it seems. What happens if two simultaneous additions happen at the same memory location?

Synchronization

Threads occasionally have to be synchronized for various reasons:

- You want to make sure that all the work has finished before progressing (to do a MPI communication perhaps)
- There is some sensitive section which can only be accessed by a single thread at a time

Ways to do this:

- `critical` marks a section that only one thread can enter at a time
- `atomic` marks a memory location that can only be updated in serial
- `barrier` waits until all threads arrive
- `ordered` executes a section in order

Locks

Locks are similar to critical sections but they are a bit more flexible.

```
omp_lock_t writelock;

omp_init_lock(&writelock);

#pragma omp parallel for
for ( i = 0; i < x; i++ ) {
    // some stuff
    omp_set_lock(&writelock);
    // one thread at a time stuff
    omp_unset_lock(&writelock);
    // some stuff
}

omp_destroy_lock(&writelock);
```

Because you are locking a data element, you can be more dynamic than just marking a section as critical.

OpenMP Runtime

There are a number of OpenMP routines that are useful during runtime:

- `omp_set_num_threads(N)`
- `omp_get_num_threads()`
- `omp_get_thread_num()`
- `omp_get_max_threads()`
- `omp_in_parallel()`
- `omp_num_procs()`

Compiling OpenMP Code

OpenMP uses compiler directives. However, unlike MPI, it doesn't provide its own compiler. Instead, when compiling OpenMP code, you have something like `g++ -fopenmp omp_code.cpp -o omp_code`. However, linking can be more complicated. On my laptop, for example, after running `brew install libomp`, to compile, the linking has to be specified.
Why do I mention this?

OpenMP GPU Offload

OpenMP lets you use GPUs if the compiler you are using supports this.

```
#pragma omp target data map(to: B[0:n], C[0:n]) map(from: A[0:n])
{
    #pragma omp target teams distribute parallel for
    for (size_t i = 0; i < n; i++) {
        A[i] = B[i] + scalar * C[i];
    }
}
```

Key elements:

- `map(to:)` indicates that the specified arrays need to be copied to the GPU
- `map(from:)`indicates that the specified array will copied to the CPU
- `target` indicates that the target device (the GPU) will be used
- `teams distribute` indicates that multiple teams of threads should be used on the GPU, and that work should be distributed among them. Without specifying teams, only one thread will be used on the GPU.

Compiler Issues

This is very nice and very easy. However...

	NVC/NVFortran	Clang/Cray/AMD	GCC/GFortran
OpenMP flag	`-mp`	`-fopenmp`	`-fopenmp -foffload=<target>`
Offload flag	`-mp=gpu`	`-fopenmp-targets=<target>`	`-foffload=<target>`
Target NVIDIA	default	`nvptx64-nvidia-cuda`	`nvptx-none`
Target AMD	n/a	`amdgcn-amd-amdhsa`	`amdgcn-amdhsa`
GPU Architecture	`-gpu=<cc>`	`-Xopenmp-target -march=<arch>`	`-foffload="-march=<arch>`

Additionally, performance issues? OpenMP is high level, so fine grained control over GPU parallelism isn't possible.

For your projects

Recall that the project is meant to be MPI + X where X is one of OpenMP, CUDA, Kokkos. If you choose OpenMP, some things to consider:

- What do you parallelize across MPI ranks, and what do you parallelize with OpenMP threads?
- What is the most effective way of synchronizing MPI ranks and OpenMP threads? How much synchronization is needed?
- How do threads interact with MPI windows?
- On Vanda, each CPU node has 72 CPU cores. Is it better to only have 1 MPI rank per node and only use OpenMP for intra-node parallelism, or is faster to have 2 MPI ranks and 36 OpenMP threads per rank, or 4 and 18, or whatever?

1 OpenMP

2 C++ Codebase Structure

3 Matrix Multiplication

Structuring Your Code

In all the code snippets I've shown, the code is usually pretty simple. How is more complicated code structured?

- Code usually lives in `/src/`
- Executables usually live in `/examples/` or `/executables/`
- Code is linked with headers

Header Files

In C++, code in other files has to be declared in header files. In the .cpp file, you put

- Function implementation
- Template instantiation

In the .hpp file, you put

- Function signatures
- Templates
- Structs and classes

The .hpp files are then included in the main executable code. When compiling, specify the linking as well.

Makefiles

Instead of doing all the linking in your command line, there is an alternative where you specify the compilation chain in a file, called a makefile. Advantages of this include:

- Clear dependencies between files
- Only recompile when code changes

However, handwriting makefiles can be unpleasant.

Cmake

Cmake is a way to automatically generate a makefile. In your codebase, you specify dependencies and relationships in CMakeLists.txt, which CMake then turns into a makefile, which you then compile. Advantages:

- Makes it easy to link external codebases
- Automates compiler detection
- Automatically detects dependencies, no need to explicitly write them out

1 OpenMP

2 C++ Codebase Structure

3 Matrix Multiplication

Matrix Multiplication

Given two $N \times N$ matrices A and B, the computation of $C = AB$ requires $2N^3$ flops. This is expensive! Let's parallelize it. First of all, let's write matrix multiplication as three nested loops:

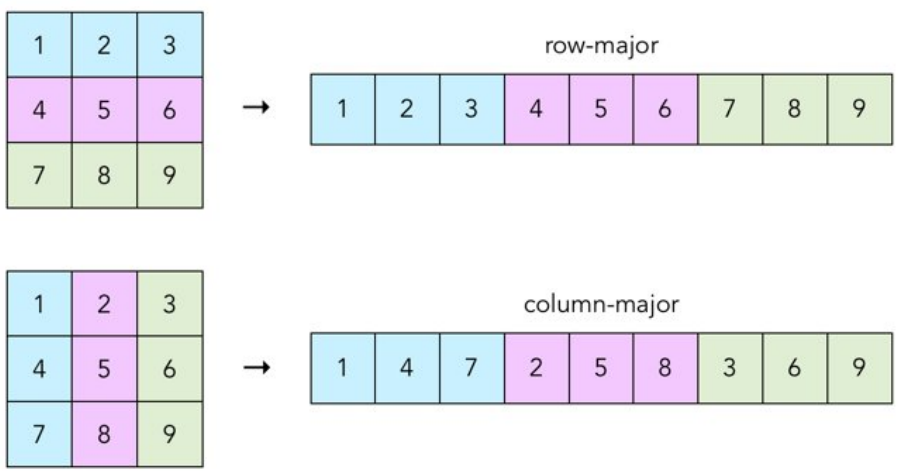
```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        for (int k = 0; k < N; k++) {
            Cmat[i,j] += Amat[i,k] * Bmat[k,j];
        }
    }
}
```

Data Layouts

Before we parallelize anything, let us first address memory locality. Looking at the matrices:

- The second index of C changes the fastest
- The second index of A changes the fastest
- The first index of B changes the fastest

We want entries that will be accessed sequentially to be adjacent in memory. Two options: row major and column major.



MPI Parallelization

Now, let's parallelize this with MPI. Some considerations:

- Matrix initialized on rank 0 so a broadcast will be needed
- Does each MPI rank need a complete copy of every matrix?
- Which loop to split up? Determines which MPI ranks need which parts of the matrix

To start, broadcast the full matrix, split up the outermost loop. Because the outermost loop is being split up, each MPI rank ends up with a subset of the C matrix which can be assembled without any further additions.

Adding OpenMP

We can further parallelize this with OpenMP. Which loop should OpenMP be applied to? How should scheduling be performed?

Tiling

When performing matrix multiplication, we can decompose our problem with tiles:

$$C = AB = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

so expanding, we find that

$$C = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

We can use this if the A and B matrices are distributed, and also to minimize cache misses.

Caching

Our matrices A, B, and C are large and live in memory. To do arithmetic, parts of the matrices are moved to the CPU cache. This means that

- We want to minimize cache misses, where data is moved between memory and the cache
- We want to maximize how much we use data living in the cache

Tiling provides a way of reusing data.

Further Considerations

This was not at all comprehensive and there's many things that could be tried as well:

- Changing the order of the loops doesn't affect the answer, but does it affect the runtime?
- Do different tile sizes behave differently for different dimensions? How does this relate to cache size?
- What if the matrices aren't square?
- Is there a way to reduce the amount of OpenMP parallel overhead?
- Do techniques that work well on one computer also work well on other computers?